

PROMPTS PARA IA E GUIA DE APRESENTAÇÃO

Material de apoio para alunos

PARTE 1: PROMPTS PRONTOS PARA USO COM IA

Prompt 1: Gerar Esqueleto de Factory

Você é um especialista em padrões de design em C# e .NET. Preciso que você gere o esqueleto completo de código para um [Factory Method / Abstract Factory / Simple Factory] pattern.

Contexto:

- Domínio: [descreva seu domínio, ex: sistema de pagamentos]
- Produtos: [liste os tipos de produtos, ex: CreditCard, Boleto, Pix]
- Framework: .NET 8
- Integração com Microsoft.Extensions.DependencyInjection

Requisitos:

1. Interfaces bem definidas com documentação XML
2. Implementações concretas com comentários explicativos em português
3. Factory com validação de entrada e tratamento de erros
4. Registro adequado no IServiceCollection
5. Exemplo de uso demonstrando o padrão

Por favor, gere código completo, compilável e seguindo as melhores práticas SOLID.

Prompt 2: Gerar Testes Unitários

Você é um especialista em testes unitários com xUnit em C#. Tenho o seguinte código de factory:

[cole seu código aqui]

Preciso que você gere uma suíte completa de testes unitários cobrindo:

1. Casos de sucesso (criação de cada tipo de produto)
2. Casos de erro (entradas inválidas, nulas, tipos não suportados)
3. Integração com DI (verificar que dependências são resolvidas)
4. Comportamento de lifetimes (Transient, Scoped, Singleton)
5. Testes usando Theory e InlineData para múltiplos cenários

Use Moq para mocking de dependências quando necessário.

Use nomenclatura descritiva seguindo padrão `MethodName_Scenario_ExpectedBehavior`.

Adicione comentários explicando o propósito de cada teste.

Prompt 3: Gerar Diagramas UML

Você é um especialista em UML e modelagem de software. Preciso que você gere diagramas PlantUML para o seguinte código:

[cole seu código aqui]

Gere:

1. Diagrama de classes mostrando:

- Todas as interfaces e classes
- Relações de herança, implementação e associação
- Visibilidade de membros (+, -, #)
- Anotações explicando o padrão usado

2. Diagrama de sequência mostrando:

- Fluxo típico de criação de objeto
- Interação entre cliente, factory e produtos
- Resolução via DI container

Forneça o código PlantUML completo e renderizável.

Prompt 4: Gerar Documentação README

Você é um technical writer especializado em documentação de software. Preciso de um README.md profissional para meu projeto de Factory Patterns.

Informações do projeto:

- Nome: [nome do projeto]
- Padrão implementado: [qual padrão]
- Domínio: [descrição do domínio]
- Tecnologias: .NET 8, xUnit, Moq

O README deve conter:

1. Título e descrição concisa do projeto
2. Badges (build status, coverage, etc)
3. Seção "Getting Started" com pré-requisitos e instalação
4. Seção "Architecture" explicando o padrão usado
5. Exemplos de código mostrando uso básico e avançado
6. Seção "Running Tests" com comandos
7. Diagrama de arquitetura (Mermaid)
8. Seção "Design Decisions" justificando escolhas
9. Contribuindo e licença

Use Markdown com formatação profissional e emojis apropriados.

Prompt 5: Gerar Slides de Apresentação

Você é um designer de apresentações técnicas. Preciso de um roteiro completo para slides sobre meu projeto de Factory Patterns.

Contexto:

- Apresentação de 10-15 minutos
- Audiência: professores e colegas de graduação
- Tema: [seu tema específico]

Gere uma estrutura de slides contendo:

Slide 1 - Título

- Título do projeto
- Seu nome
- Data

Slide 2 - Problema

- Qual problema o projeto resolve
- Cenário de exemplo concreto

Slide 3 - Solução Proposta

- Padrão escolhido e por quê
- Visão geral da arquitetura

Slides 4-6 - Arquitetura Detalhada

- Diagrama de classes
- Diagrama de sequência
- Integração com DI

Slides 7-9 - Demonstração

- Código principal
- Exemplo de uso
- Resultados de testes

Slide 10 - Design Decisions

- Decisões importantes tomadas
- Trade-offs considerados

Slide 11 - Lições Aprendidas

- Desafios enfrentados
- O que você aprendeu

Slide 12 - Conclusão e Q&A

- Resumo
- Próximos passos

- Agradecimentos

Para cada slide, forneça:

- Título sugerido
- Pontos principais (bullet points)
- Notas do apresentador
- Sugestões visuais

Prompt 6: Refatorar Código Legacy

Você é um especialista em refatoração de código. Tenho o seguinte código legacy que precisa ser refatorado para usar Factory Pattern com DI:

[cole código legacy aqui]

Problemas identificados:

- Instanciação direta com 'new'
- Acoplamento forte
- Difícil de testar
- Não usa DI

Por favor:

1. Identifique todos os problemas de design
2. Proponha qual padrão Factory seria mais adequado
3. Refatore o código aplicando o padrão
4. Mostre como configurar DI
5. Crie testes comparando comportamento antes/depois
6. Explique os benefícios da refatoração

Use comentários // ANTES: e // DEPOIS: para destacar mudanças importantes.

Prompt 7: Code Review Automatizado

Você é um revisor de código sênior especializado em C# e padrões de design. Por favor, revise o seguinte código de factory pattern:

[cole seu código aqui]

Analise os seguintes aspectos:

1. SOLID Principles
 - Há violações de SRP, OCP, LSP, ISP ou DIP?
2. Implementação do Pattern
 - O padrão está implementado corretamente?

- Há anti-patterns presentes?

3. Integração com DI

- Lifetimes são apropriados?
- Há uso de Service Locator anti-pattern?

4. Qualidade de Código

- Nomenclatura clara?
- Tratamento de erros adequado?
- Validação de entrada presente?

5. Testabilidade

- Código é facilmente testável?
- Interfaces permitem mocking?

6. Performance

- Há problemas de performance óbvios?
- Cache deveria ser usado?

Para cada problema encontrado, forneça:

- Severidade (Crítico/Alto/Médio/Baixo)
- Descrição do problema
- Código sugerido para correção
- Justificativa da mudança

Ao final, dê uma nota geral de 0-10 e um resumo executivo.

Prompt 8: Gerar Casos de Teste Adicionais

Você é um QA engineer especializado em testes de software. Analisando o seguinte código de factory e seus testes existentes:

[cole código e testes existentes]

Identifique casos de teste que estão faltando. Considere:

1. Edge cases (valores limite, casos extremos)
2. Testes de concorrência (thread-safety)
3. Testes de performance (se factory é chamado frequentemente)
4. Testes de integração (interação com outros componentes)
5. Testes de contrato (verificar que interfaces são respeitadas)
6. Testes negativos (comportamento com dependências indisponíveis)

Para cada caso de teste identificado:

- Nomeie o teste
- Descreva o cenário

- Forneça o código completo do teste
- Explique por que esse teste é importante

PARTE 2: GUIA DE APRESENTAÇÃO ORAL

Estrutura Recomendada (10-15 minutos)

Minutos 0-2: Introdução e Contextualização

Comece apresentando o problema que seu projeto resolve de forma clara e concreta. Use um exemplo do mundo real que a audiência possa relacionar. Por exemplo, se seu projeto é sobre processamento de pagamentos, comece dizendo algo como: "Imagine que você está desenvolvendo um e-commerce que precisa aceitar cartão de crédito, boleto e PIX. Como você estruturaria o código para que seja fácil adicionar novos métodos de pagamento no futuro sem modificar código existente?"

Estabeleça por que o problema é importante e mencione brevemente qual solução você implementou (qual padrão Factory). Não entre em detalhes técnicos ainda, apenas crie o contexto e desperte interesse.

Minutos 2-4: Visão Geral da Solução

Apresente o padrão Factory que você escolheu e explique rapidamente por que ele é adequado para o problema. Use um diagrama de alto nível mostrando a arquitetura geral da sua solução. Este é o momento de mostrar o "big picture" antes de mergulhar nos detalhes.

Mencione as tecnologias principais que você usou: .NET 8, Dependency Injection, xUnit para testes. Isso estabelece o contexto técnico para o resto da apresentação.

Minutos 4-8: Demonstração Técnica

Esta é a parte central da sua apresentação. Mostre o código principal do seu factory pattern. Não mostre todo o código linha por linha, mas destaque as partes mais importantes. Use slides com snippets de código bem formatados e comentados.

Mostre o diagrama de classes explicando as relações entre interfaces, implementações concretas e o factory. Depois mostre o diagrama de sequência explicando o fluxo de execução quando um cliente solicita a criação de um objeto.

Demonstre a integração com Dependency Injection. Mostre como os serviços são registrados no container e como o factory resolve as dependências automaticamente. Este é um diferencial importante que mostra maturidade técnica.

Se possível, execute o código ao vivo mostrando a aplicação funcionando. Uma demonstração prática vale mais que slides teóricos. Se executar ao vivo for arriscado, grave um vídeo curto (1-2 minutos) mostrando o sistema em ação.

Minutos 8-10: Testes e Qualidade

Mostre que seu código tem testes. Apresente alguns testes unitários representativos e execute a suíte de testes mostrando que todos passam. Mencione a cobertura de código se for alta.

Explique como os testes garantem que o factory funciona corretamente e como eles facilitam futuras manutenções. Destaque testes de casos de erro, mostrando que você pensou não apenas no caminho feliz mas também em situações excepcionais.

Minutos 10-12: Decisões de Design e Trade-offs

Esta seção demonstra pensamento crítico. Explique decisões importantes que você tomou durante o desenvolvimento. Por exemplo: "Escolhi Singleton para o factory porque ele não mantém estado e é thread-safe, o que melhora performance ao evitar criações desnecessárias".

Mencione trade-offs considerados. Por exemplo: "Considerei usar Simple Factory por ser mais simples, mas optei por Factory Method porque o sistema precisa ser facilmente extensível para novos tipos de produtos". Isso mostra que você avaliou múltiplas opções e fez escolhas informadas.

Minutos 12-13: Lições Aprendidas

Seja honesto sobre desafios enfrentados e o que você aprendeu. Por exemplo: "Inicialmente eu estava usando Service Locator anti-pattern sem perceber. Após estudar mais sobre DI, refatorei para usar Constructor Injection, o que melhorou muito a testabilidade".

Mencione conceitos que você dominou durante o projeto: "Antes deste projeto, eu usava Dependency Injection de forma superficial. Agora entendo profundamente lifetimes, resolução de dependências e como factories se integram com DI containers".

Minutos 13-14: Conclusão

Recapitule brevemente os pontos principais: problema, solução, padrão usado, benefícios alcançados. Mencione possíveis extensões futuras do projeto.

Agradeça a atenção da audiência e abra para perguntas.

Minutos 14-15: Perguntas e Respostas

Esteja preparado para perguntas comuns como:

- "Por que você escolheu este padrão específico em vez de outro?"
 - "Como seu sistema se comportaria com X cenário?"
 - "Quais foram os maiores desafios?"
 - "Como você garantiu que o código é testável?"
-

PARTE 3: DICAS DE APRESENTAÇÃO

Preparação

Pratique várias vezes: Ensaie sua apresentação pelo menos três vezes completas. Cronometre para garantir que está dentro do tempo limite. Pratique falando em voz alta, não apenas mentalmente.

Prepare demos com backup: Se você vai executar código ao vivo, tenha screenshots ou vídeo gravado como backup caso algo dê errado. Murphy's Law é real em demos ao vivo.

Conheça profundamente seu código: Você deve ser capaz de explicar qualquer linha do seu código. Se durante a apresentação alguém perguntar sobre uma parte específica, você precisa saber responder com confiança.

Antecipe perguntas: Pense nas perguntas mais difíceis que poderiam ser feitas e prepare respostas. Melhor estar super-preparado que ser pego de surpresa.

Durante a Apresentação

Comece com confiança: Os primeiros 30 segundos estabelecem o tom. Respire fundo, fale claramente, faça contato visual com a audiência.

Use linguagem clara: Evite jargões desnecessários. Se usar um termo técnico, explique brevemente o que significa. Lembre-se que nem todos na audiência têm o mesmo nível de conhecimento.

Conte uma história: Em vez de apenas listar features, conte a história do desenvolvimento: "Comecei com uma implementação simples, depois percebi que tinha problemas de extensibilidade, então refatorei para usar Factory Method, o que resolveu o problema elegantemente".

Mostre entusiasmo: Você passou horas trabalhando neste projeto. Deixe sua paixão e orgulho transparecerem. Entusiasmo é contagiante e torna a apresentação mais interessante.

Interaja com a audiência: Faça perguntas retóricas ocasionalmente: "Vocês conseguem imaginar o que aconteceria se tentássemos adicionar um novo tipo de pagamento neste código?". Isso mantém a audiência engajada.

Gerencie o tempo: Tenha um relógio visível. Se estiver ficando sem tempo, pule partes menos importantes. Nunca passe do tempo limite estabelecido.

Lide com nervosismo: É normal estar nervoso. Respire profundamente, fale devagar, beba água se precisar. O nervosismo diminui após o primeiro minuto.

Slides Efetivos

Regra 10-20-30: No máximo 10 slides (para 15 minutos), apresente em 20 minutos ou menos (deixando tempo para perguntas), use fonte tamanho 30+ para texto principal.

Uma ideia por slide: Cada slide deve comunicar uma ideia principal. Slides poluídos confundem a audiência.

Código legível: Use fonte grande (18+) para código. Destaque linhas importantes com cor ou setas. Remova código boilerplate desnecessário.

Diagramas claros: Diagramas devem ser simples e focar no que você está explicando naquele momento. Não mostre um diagrama gigante com 20 classes se está falando sobre 3.

Evite parágrafos: Use bullet points curtos. Você deve falar os detalhes, não colocar tudo no slide.

Contraste alto: Texto escuro em fundo claro ou texto claro em fundo escuro. Evite combinações de baixo contraste que são difíceis de ler.

PARTE 4: CHECKLIST DE APRESENTAÇÃO

Uma Semana Antes

- ☐ Slides completos e revisados
- ☐ Código funcionando perfeitamente
- ☐ Testes todos passando
- ☐ Demo preparada (ao vivo ou vídeo)
- ☐ Notas de apresentação escritas
- ☐ Ensaio completo realizado

Três Dias Antes

- ☐ Segundo ensaio completo
- ☐ Feedback de colega ou professor obtido
- ☐ Ajustes baseados em feedback realizados
- ☐ Perguntas potenciais identificadas e respostas preparadas
- ☐ Material de backup pronto

Um Dia Antes

- ☐ Terceiro ensaio completo cronometrado
- ☐ Equipamento testado (projetor, laptop, clicker)
- ☐ Arquivos sincronizados na nuvem (backup)
- ☐ Roupas apropriadas separadas
- ☐ Boa noite de sono planejada

Dia da Apresentação

- ☐ Chegue cedo para setup
- ☐ Teste equipamento no local
- ☐ Respire fundo e relaxe

- ☐ Revise mentalmente pontos principais
 - ☐ Hidrate-se
 - ☐ Desligue notificações do celular
 - ☐ Apresente com confiança!
-

PARTE 5: AUTOAVALIAÇÃO PÓS-APRESENTAÇÃO

Após sua apresentação, reflita sobre estas questões:

Conteúdo Técnico:

- Demonstrei domínio completo do padrão Factory implementado?
- Expliquei claramente a integração com Dependency Injection?
- Mostrei testes funcionais e de qualidade?
- Justifiquei minhas decisões de design adequadamente?

Habilidades de Apresentação:

- Mantive contato visual com a audiência?
- Falei com clareza e volume adequado?
- Gerenciei bem o tempo disponível?
- Demonstrei entusiasmo e confiança?
- Respondi perguntas de forma completa e profissional?

Materiais Visuais:

- Slides eram claros e profissionais?
- Código estava legível e bem formatado?
- Diagramas comunicavam efetivamente as ideias?
- Demonstração funcionou conforme planejado?

Pontos de Melhoria:

- O que faria diferente em uma próxima apresentação?
- Quais perguntas me pegaram desprevenido?
- Onde poderia ter sido mais claro ou conciso?
- Que feedback recebi que posso aplicar no futuro?

Esta reflexão é valiosa para seu crescimento profissional. Apresentações técnicas são uma habilidade essencial na carreira de desenvolvedor.

RECURSOS ADICIONAIS

Ferramentas Úteis

Para Slides:

- Google Slides (colaborativo, acessível)
- Microsoft PowerPoint (profissional, rico em recursos)
- Reveal.js (slides em HTML para desenvolvedores)

Para Diagramas:

- PlantUML online: <https://www.plantuml.com/plantuml/>
- Mermaid Live: <https://mermaid.live/>
- Draw.io: <https://app.diagrams.net/>

Para Gravação de Demo:

- OBS Studio (gratuito, poderoso)
- ScreenToGif (simples para GIFs)
- Loom (fácil de usar, baseado na web)

Para Prática:

- Grave você mesmo apresentando e assista depois
- Apresente para amigos/família primeiro
- Use aplicativos de teleprompter se necessário

Leituras Complementares

Para aprofundar conhecimento antes da apresentação:

- "Design Patterns" (Gang of Four) - capítulos sobre Factory
- "Dependency Injection Principles" (Mark Seemann)
- Documentação oficial Microsoft sobre DI
- Blog posts sobre padrões Factory em C#

Boa sorte com sua apresentação! Lembre-se: você trabalhou duro neste projeto e tem muito conhecimento para compartilhar. Confie em si mesmo e aproveite a oportunidade de mostrar o que aprendeu.